

TinyML Gesture Reader

Gestusbaseret blackjack-simulation på Particle Photon 2

Kursus	ETTML-01 Tiny Machine Learning
Studerende	Erik Kjær Klint
Studienummer	201704536 / AU591167
Platform	Particle Photon 2 + ADXL343
Afleveringsdato	10. august 2026
Repository	ETTML—Gesture-Analysis

Semester-/eksamensprojekt

Indholdsfortegnelse

1	Resumé	3
2	Introduktion	3
3	Projektbeskrivelse og scope	3
3.1	Brug af AI-værktøjer	4
4	Kravanalyse	4
4.1	Sporbarhed til kursuskrav	4
4.2	Funktionelle og ikke-funktionelle mål	4
5	Systemdesign	4
5.1	Dataflow og ansvar	4
5.2	Hardware og sensor	5
5.3	Driftsformer og protokol	5
6	Dataindsamling	6
6.1	Kalibrering og iterativ metode	6
6.2	Endeligt datasæt	6
7	Preprocessing og model	7
7.1	Featurekontrakt	7
7.2	MLP og eksport	8
8	Firmwareimplementering	8
8.1	Målinger og modelberegning	8
8.2	LED-controller	8
9	Test og resultater	8
9.1	Offline evaluering	8
9.2	Embedded og LIVE-verifikation	9
9.3	Verifikationsstatus	9
10	Diskussion	10
11	Konklusion	10
12	Fremtidigt arbejde	10
13	Referencer	10
14	Appendiks A: Reproduktion	11
15	Appendiks B: Kodearkitektur og sporbarhed	11
16	Appendiks C: Datakontrakt	11
17	Appendiks D: Værktøjer og AI-assistance	11

1 Resumé

Projektet er en lille gestuslæser, som kører direkte på en Particle Photon 2. En ADXL343 måler bevægelse på tre akser 400 gange i sekundet. Fire sekunders målinger bliver kogt ned til 28 tal, som beskriver signalet. De sendes gennem et lille neuralt netværk med lagene 28–32–16–5. Modellen vælger mellem fem tilstande: ingen bevægelse, ét, to eller tre tryk samt en rystelse fra side til side. De fire aktive gestusser svarer til blackjack-kommandoerne stand, hit, exit og split.

Det endelige datasæt består af 25 godkendte optagelser, fem fra hver klasse. Programmet valgte instruktionerne i tilfældig rækkefølge og afviste selv et forsøg, hvis antallet af tryk var forkert, hvis en idle-optagelse indeholdt bevægelse, eller hvis en rystelse var for svag. To forsøg blev afvist og taget om. Alle godkendte filer har 1.600 målepunkter i rigtig rækkefølge, uden manglende værdier eller målinger uden for sensorens område.

Data blev delt, så 80 % blev brugt til træning og 20 % til test. Modellen ramte rigtigt i fire ud af fem testeksempler, altså 80 %. Det opfylder projektets mål, men fem eksempler er alt for få til at bevise, at modellen virker lige godt for andre brugere. På Photon 2 tog selve modelberegningen i gennemsnit 345 μ s og højst 364 μ s. Firmwaren brugte 27.950 B flash og 46.686 B RAM. LIVE blev også prøvet på den fysiske enhed, hvor de korrekte LED-mønstre blev vist. Resultatet er derfor en fungerende prototype, men ikke et færdigt forbrugerprodukt.

2 Introduktion

TinyML handler kort sagt om at få machine learning til at køre på små computere tæt på sensoren. På kurset arbejder vi blandt andet med gode datasæt, behandling af sensordata og med at få en trænet model over på en microcontroller [1]. Semesterprojektet kræver en Photon 2, mindst én sensor, egne mærkede data, passende databehandling og en ML- eller DL-model med et resultat, man kan se eller aflæse [2].

Gestusgenkendelse passer godt som klassifikationsopgave. Målingerne ændrer sig både med antallet af tryk, hastigheden, kraften, monteringen og sensorens retning. En fast grænse kan godt opdage et hårdt slag, men den har sværere ved at skelne sikkert mellem alle fem klasser. Derfor blev projektets hovedspørgsmål:

Kan en lille model på Photon 2 skelne mellem ingen bevægelse, ét, to og tre tryk samt en rystelse fra side til side—og bagefter vise resultatet tydeligt uden at bruge for meget tid eller hukommelse?

Projektet er udført individuelt af Erik Kjær Klint. Jeg har stået for opkobling, dataindsamling, Python-kode, træning, firmware, test og rapport. AI-værktøjerne BlackBox AI og OpenAI Codex er brugt som hjælp til kode, fejlsøgning, struktur og sproglig redigering [10], [11]. Jeg har selv udført forsøgene og kontrolleret resultaterne mod data, kildekode, build-output og den fysiske enhed.

3 Projektbeskrivelse og scope

Projektet er en gestusgrænseflade til en blackjack-simulation og ikke et helt blackjack-spil. Når systemet har genkendt en gestus, sender det en kommando og viser et LED-mønster. På den måde kan projektet holde fokus på sensoren, dataene, modellen og den indbyggede løsning.

Klasse	Kommando	LIVE-feedback
idle	ingen	LED slukket
tap1	stand	blå i 1,0 s
tap2	hit	to blå pulser med 0,50 s startinterval
tap3	exit	tre røde pulser med ca. 0,33 s startinterval
shake_lr	split	rød-blå-rød-blå med 1,0 s startinterval

Firmwaren har tre tydelige tilstande. DEBUG bruges til at se målinger og finde fejl. TRAINING bruges, når der skal samles data. LIVE bruger modellen og reagerer på gestusser med serielle beskeder og LED-lys. I LIVE

bliver et resultat først godkendt, når modellen er sikker og har set samme resultat flere gange. Tap-klasserne får desuden kontrolleret det målte antal slag.

3.1 Brug af AI-værktøjer

BlackBox AI blev brugt tidligt i projektet til forslag til kode og fejlsøgning. OpenAI Codex blev senere brugt til at gennemgå projektet, rette kode, analysere data, hjælpe med dokumentation og gøre rapporten klar [10], [11]. AI har altså været et hjælpeværktøj på samme måde som en avanceret kodeassistent. Det har ikke udført de fysiske forsøg eller leveret måleresultaterne. Jeg har gennemgået ændringerne, kørt testene og taget de endelige valg. Denne oplysning står både her og i appendiks D, så brugen er tydelig.

4 Kravanalyse

4.1 Sporbarhed til kursuskrav

Semesterbeskrivelsen kræver som minimum introduktion, projektbeskrivelse, kravanalyse, systemdesign, implementering, test og konklusion. Den anbefaler omkring 20 sider [2]. Tabellen herunder viser, hvor hvert teknisk krav bliver opfyldt.

Krav	Implementering/evidens	Status
Photon 2 embedded hardware	Product/firmware/src/main.cpp; fysisk LIVE-test	Opfyldt
Lokal sensor	ADXL343 via I2C; DEVID- og read-error-kontrol	Opfyldt
Egne mærkede data	25 accepterede v3-CSV'er med labels og metadata	Opfyldt
Behandling af målinger	fjernelse af gennemsnit, samlet bevægelse og 28 egenskaber	Opfyldt
ML-/DL-algoritme	StandardScaler + MLP 28-32-16-5	Opfyldt
Lokal forudsigelse	C++ forward-pass og softmax på Photon 2	Opfyldt
Observerbar tilstand	STATUS, EVENT og RGB-mønstre	Opfyldt
Delbar kode/data/dokumentation	GitHub-repository med kildekode, CSV og PDF	Klargjort
Hardwaredokumentation	pin-tabel, foto og officielle databladreferencer	Opfyldt i rapport

4.2 Funktionelle og ikke-funktionelle mål

Systemet skal måle stabilt, vælge mellem fem klasser, undgå at reagere under idle og vise den rigtige LED-kode. Mine egne mål var mindst 80 % på testdata, under 50 ms til selve modelberegningen og et program, som uden problemer kan være på Photon 2. Det er mine projektmål og ikke krav fra kurset.

Photon 2 har ifølge Particle en 200 MHz Arm Cortex-M33, op til 2 MB plads til brugerprogrammet og 3 MB RAM [4]. Der er derfor god plads til modellen og bufferen med 1.600 målinger, selv om det stadig er en god idé at holde løsningen lille og enkel.

5 Systemdesign

5.1 Dataflow og ansvar

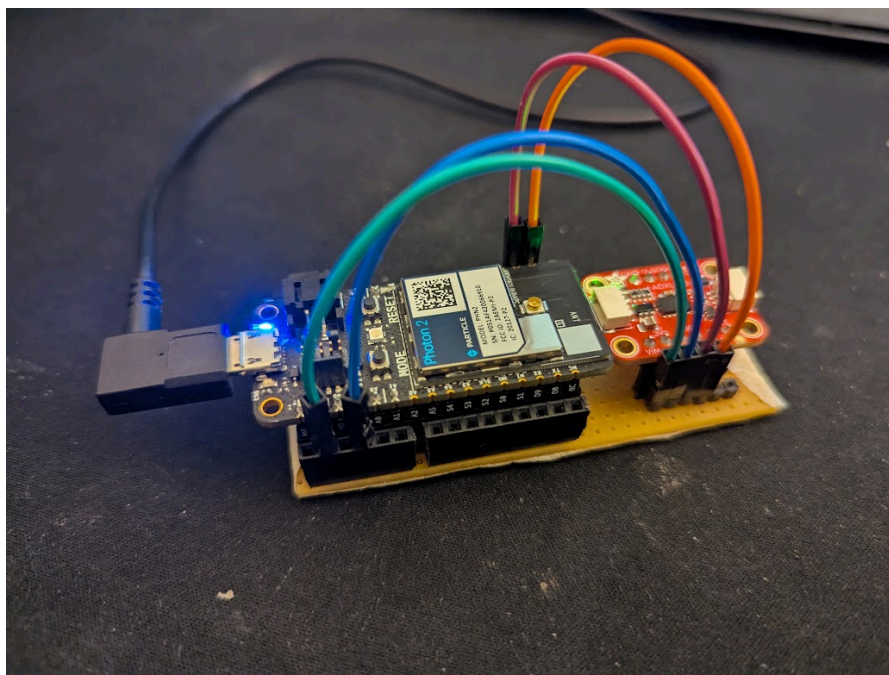
ADXL343400 Hz XYZ	→	4 s vindue 1.600 målinger	→	28 egenskaber
		↓		
RGB + EVENT	←	Beslutningsfilter	←	Scaler + MLP

Tabel 1: Dataens vej gennem systemet. De samme 28 egenskaber beregnes under træning og på Photon 2.

Først læses X, Y og Z samtidig fra sensoren. Derefter fjernes den faste hældning i hver kanal, den samlede bevægelsesstyrke beregnes, og syv egenskaber findes for hver af de fire signaler. Modellen får dermed 28 tal at arbejde med. Før en kommando bliver sendt, skal samme sikre resultat komme tre gange i træk. Systemet venter også fire sekunder, før den samme bevægelse kan udløse en ny kommando. Til sidst viser LED'en resultatet, og det bliver skrevet over USB.

5.2 Hardware og sensor

ADXL343 kan måle i områderne ± 2 , ± 4 , ± 8 og ± 16 g. I full-resolution svarer ét trin omtrent til 3,9 mg, og sensoren kan levere op til 3.200 målinger i sekundet [3]. Her bruges ± 16 g og 400 Hz. Databladet siger, at støjen vokser lidt over 100 Hz, men 400 Hz er stadig inden for sensorens normale område [3]. Den højere hastighed blev nødvendig, fordi et tap sker meget hurtigt. Ved de første forsøg med 50 Hz kunne vigtige dele af slaget ganske enkelt ligge mellem to målinger.



Figur 1: Fysisk prototype med Photon 2, ADXL343 og lokal RGB-feedback.

Photon 2	ADXL343	Funktion
3V3	VCC	3,3 V forsyning
GND	GND	fælles reference
D0 / SDA	SDA	I2C-data
D1 / SCL	SCL	I2C-clock

Ved opstart prøver firmwaren begge mulige sensoradresser, 0x53 og 0x1D, og kontrollerer ID-værdien 0xE5. Hvis noget går galt, kan det ses med kommandoen STATUS. Hver måling får et tidsstempel i mikrosekunder. I den sidste serie lå det normale mellemrum mellem målingerne på 2.498–2.501 μ s, hvilket ligger meget tæt på de ønskede 400 Hz.

5.3 Driftsformer og protokol

Enheden styres med enkle tekstkommandoer som MODE DEBUG, MODE TRAINING, MODE LIVE, STATUS og TAP_SCOPE. I LIVE sendes en besked som EVENT,class=tap2,command=hit,score=..., når en gestus er godkendt. LED-koden bruger ikke ventekald, så sensoren kan fortsætte med at måle, mens lyset blinker.

6 Dataindsamling

6.1 Kalibrering og iterativ metode

De første optagelser ved 50 Hz viste, at hurtige tryk kunne blive overset. Derfor lavede jeg en lille oscilloskop-test ved 400 Hz. En gul nedtælling varer tre sekunder, hvorefter programmet optager et halvt sekund før det grønne GO-signal og 3,5 sekunder bagefter. Testen viste også, at mine egne beskrivelser “light”, “normal” og “firm” ikke gav tre tydeligt adskilte kraftniveauer. De bruges derfor kun til at få mere variation i dataene og ikke som facit for kraften.

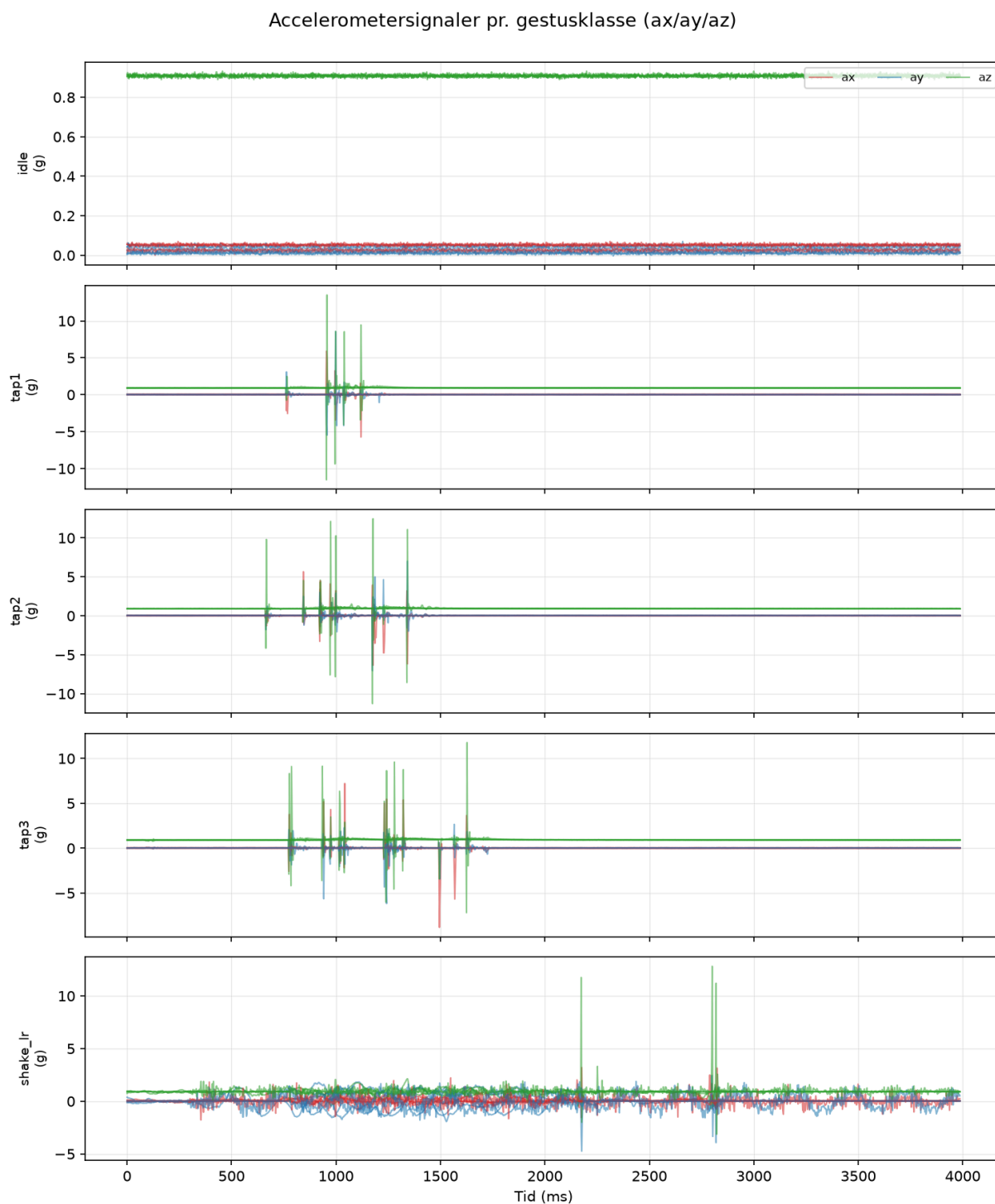
Optagelsesvinduet vælger klasse, tempo og ønsket kraft i tilfældig rækkefølge. Instruktionen vises med stor tekst, mens skærmen er gul. Når den bliver grøn, udføres bevægelsen, og en timer viser den resterende tid. Programmet kontrollerer bagefter, om sensoren gik uden for sit område, om idle faktisk var stille, om antallet af tryk passede, og om en rystelse varede længe nok. Et dårligt forsøg gemmes som afvist og bliver automatisk prøvet igen senere.

6.2 Endeligt datasæt

Den session, som modellen blev trænet på, hedder 20260810_141717. Den indeholder fem godkendte forsøg fra hver klasse, altså 25 i alt. Hver CSV-fil har 1.600 målinger samt klasse, ønsket tempo og kraft, sessionsnummer og godkendelsesstatus. Ved siden af ligger en JSON-fil med kontroltal og en PNG-graf, så optagelsen også kan ses med øjnene.

Klasse	Accepteret	Kvalitetskontrol
idle	5	0 bevægelseshændelser; lav spidsværdi
tap1	5	præcis 1 målt slag
tap2	5	præcis 2 målte slag
tap3	5	præcis 3 målte slag
shake_lr	5	tilstrækkelig RMS og aktiv varighed
I alt	25	lige mange fra alle fem klasser

To forsøg blev automatisk afvist. I det ene blev en idle-optagelse forstyrret af 0,1248 g bevægelse. I det andet blev der målt to slag, selv om opgaven kun bad om ét. Begge blev taget om. Alle 27 rå forsøg havde præcis 1.600 rækker, korrekt tidsrækkefølge, ingen tomme værdier og ingen målinger uden for sensorens område. Det største enkelte hul mellem to målinger var 13,976 ms, men den normale timing var stadig korrekt, og antallet af tryk kunne stadig aflæses.



Figur 2: De fem godkendte optagelser fra hver klasse i den endelige 400 Hz-session.

7 Preprocessing og model

7.1 Featurekontrakt

For hvert signal x trækkes gennemsnittet først fra: $x'_i = x_i - \bar{x}$. Den samlede bevægelsesstyrke er $m_i = (a_x^2 + a_y^2 + a_z^2)^{\frac{1}{2}}$. Derefter beregnes syv egenskaber for X, Y, Z og den samlede bevægelse:

Feature	Betydning
standardafvigelse	typisk dynamisk spredning
minimum / maksimum	negative og positive ekstremer

range	maksimum minus minimum
energi	$\frac{1}{N} \sum_i (x'_i)^2$
peak count	lokale peaks over 0,05 g med otte-sample refractory
max abs diff	største forskel mellem to nabomålinger; hvor brat slaget er

Fire signaler gange syv egenskaber giver 28 inputtal. Gennemsnittet er ikke med som egenskab, fordi det allerede er fjernet og derfor næsten altid ville være nul. `StandardScaler` sørger for, at meget store tal ikke automatisk fylder mere end små tal. Den lærer kun sine værdier fra træningsdata og bruger formelen $z = \frac{x-u}{s}$, som beskrevet i scikit-learns dokumentation [5]. De samme værdier kopieres med over i C++.

7.2 MLP og eksport

Modellen er scikit-learns `MLPClassifier`. Den har to skjulte lag med 32 og 16 neuroner og til sidst fem output –ét for hver klasse. Det er et lille neuralt netværk, som kan lære sammenhænge, der ikke kan beskrives med én lige linje. Scikit-learn træner modellen med backpropagation og bruger softmax til at lave fem sammenlignelige classesandsynligheder [6]. Træningen bruger learning rate 0,001, seed 42 og højst 500 gennemløb.

`export_model.py` træner modellen og skriver både skalering, klassenavne og vægte ud i et format, som firmwaren kan bruge. Hvis træningen fejler, stopper programmet i stedet for at overskrive en fungerende model med tomme værdier. Filerne bliver først erstattet, når hele eksporten er lykkedes. På Photon 2 ligger tallene i faste arrays, så modelberegningen ikke skal bede om ekstra hukommelse undervejs.

8 Firmwareimplementering

8.1 Målinger og modelberegning

LIVE gemmer 1.600 målinger med cirka 2.500 μ s mellem hver. Når de første fire sekunder er fyldt, beholdes de nyeste 1.500 målinger. Der skal derfor kun komme 100 nye, før modellen kan regne igen, hvilket svarer til hvert kvarte sekund. Både Python og firmwaren gør tingene i samme rækkefølge: samlet bevægelsesstyrke, fjernelse af gennemsnit, 28 egenskaber, skalering og til sidst det neurale netværk.

Et modelresultat bliver ikke brugt med det samme. Hvis sikkerheden er under 0,75, sker der ingenting. Den samme sikre klasse skal også komme tre gange i træk. De første test viste, at modellen især kunne blande ét, to og tre tryk sammen. Derfor tæller firmwaren de tydeligt adskilte slag, efter modellen har vurderet, at signalet er et tap. Det er en kombination af ML og en enkel sikkerhedsregel: modellen skelner mellem idle, tap og shake, mens reglen hjælper med antallet af tryk. Reglen beskrives åbent, fordi kursets opgave ikke må ende som ren regelprogrammering [2].

8.2 LED-controller

LED-styringen bruger ikke `delay()`. I stedet gemmer den farver, tider og antal blink og tager ét lille skridt for hver tur gennem programmets hovedløkke. Derfor kan sensoren fortsat måle 400 gange i sekundet, mens LED'en blinker.

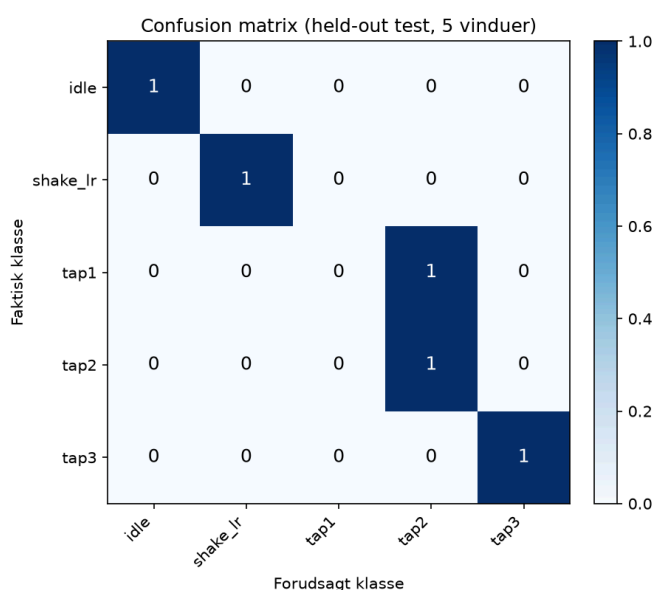
9 Test og resultater

9.1 Offline evaluering

Data blev delt på samme måde hver gang ved hjælp af seed 42. Der blev brugt 20 vinduer til træning og fem til test –ét fra hver klasse. Testen viser, om hele kæden virker, men den er for lille til at sige præcist, hvor godt modellen vil klare helt nye personer og situationer.

Metrik	Resultat	Vurdering
Accuracy	80,0 %	projekt mål nået
Macro precision	70,0 %	under mål
Macro recall	80,0 %	mål nået
Macro F1	73,3 %	under mål

Testvinduer	5	meget lille
-------------	---	-------------



Figur 3: Confusion matrix for den aktuelle 80/20-split. Fire af fem testvinduer er korrekte; tap1 forveksles med tap2.

Idle, shake, tap2 og tap3 blev rigtige, mens tap1 blev gættet som tap2. Når testen kun har fem eksempler, ændrer én fejl resultatet med 20 procentpoint. Derfor skal 80 % ikke læses som et stærkt bevis. En ekstra femdel krydsvalidering gav 76 %. Her var idle og shake stadig de letteste, mens tap-klasserne gav fejlene. Begge test peger altså i samme retning.

9.2 Embedded og LIVE-verifikation

Particle cloud compile af det endelige image lykkedes. Build-output og efterfølgende STATUS gav:

Måling	Resultat	Status
Firmware flash	27.950 B	passer
Firmware RAM	46.686 B	passer
Inference mean	345 μ s	< 50 ms
Inference max	364 μ s	< 50 ms
Sensor read errors	0	pass
Mode	LIVE	pass

Efter firmwaren var lagt på enheden, viste STATUS, at den var i LIVE, at sensoren virkede, og at der ikke var læsefejl. Jeg prøvede gestusser på den fysiske enhed og så de forventede LED-mønstre. Dermed er hele forløbet fra bevægelse til model og lys testet. Jeg nåede ikke en blind live-test med et fast antal tilfældige forsøg, så rapporten angiver ikke et tal for live-præcision.

9.3 Verifikationsstatus

Test	Status	Evidens/begrænsning
CSV-filer og klasser	Godkendt	alle 27 rå forsøg kontrolleret
Samplingkadence	Pass	median 2.498–2.501 μ s
Femklassetest på computer	Godkendt	80 % på 5 eksempler
Compile/flash	Pass	cloud build + flash success
On-device latency	Pass	345/364 μ s mean/max
LED-mapping	Pass	fysisk LIVE-kørsel

Lang idle-test for fejludløsninger	Ikke målt	kræver en længere tidsbestemt test
Flere brugere og optagedage	Ikke målt	kun én person og én slut-session
Power/endurance	Ikke målt	uden for dagens slutttest

10 Diskussion

Det vigtigste resultat er ikke kun tallet 80 %. Det vigtigste er, at hele kæden faktisk virker: sensoren måler, data bliver mærket, de 28 egenskaber beregnes, modellen kører i C++, og en godkendt gestus giver lys og en seriel besked. Beregningstiden på omkring 0,35 ms viser også, at Photon 2 har rigeligt med kræfter til denne model.

Den største svaghed er mængden af data. Fem forsøg pr. klasse fra én person på samme dag siger ikke meget om andre personer, andre dage eller en lidt anderledes placering af sensoren. Trænings- og testdata kommer også fra samme session, så resultatet kan se bedre ud, end det ville gøre i virkeligheden. En bedre test ville bruge mindst tre forskellige sessioner og gemme en hel session til den endelige test.

400 Hz gør de hurtige slag langt tydeligere, men fire sekunders buffer betyder også, at systemet først skal fyldes op, og at en gammel gestus bliver i dataene et stykke tid. Pausen på fire sekunder forhindrer dobbeltregistrering, men gør samtidig systemet langsommere at bruge. En senere version kunne starte et kortere vindue, når der opdages bevægelse.

Kombinationen af model og slag-tæller er et praktisk valg. Modellen tager sig af det samlede bevægelsesmønster, mens tælleren hjælper med forskellen mellem ét, to og tre slag. Tælleren kan ikke alene skelne sikkert mellem idle, tap og shake i alle situationer, så projektet er stadig en ML-løsning og ikke blot nogle få faste regler.

11 Konklusion

Projektet opfylder kursets vigtigste krav: Photon 2, en lokal ADXL343-sensor, egne mærkede data, behandling af signalet, en trænet femklasses MLP, beregning direkte på enheden og synligt output over USB og LED. Det endelige datasæt har 25 godkendte 400 Hz-optagelser. Testen gav 80 %, firmwaren brugte 27.950 B flash og 46.686 B RAM, og selve modellen tog omkring 0,35 ms. LIVE blev bygget, lagt på enheden og prøvet fysisk.

Resultatet er en fungerende prototype, men ikke et bevis på, at alle brugere kan få samme resultat. Jeg har vist, at hele systemet kan bygges og køre på Photon 2. Hvis løsningen skulle udvikles videre, skulle der samles mere data på forskellige dage og fra flere personer. Der skulle også laves en længere test af fejludløsninger og stabilitet.

12 Fremtidigt arbejde

De næste forbedringer ville være: flere brugere og optagedage, en test på en helt ny session, en længere idle-test, en direkte sammenligning af Python- og C++-beregninger og et kortere vindue, som starter ved bevægelse. Derefter kunne BLE eller en ekstra sensor overvejes. Modellen behøver ikke kvantisering for at kunne være på Photon 2, men det kunne være et interessant optimeringsforsøg.

13 Referencer

- [1] Aarhus Universitet. (2026). **Welcome to TinyML—ETTML-01 Tiny Machine Learning** [kursusmateriale]. Brightspace, lokal PDF-kopi.
- [2] Aarhus Universitet. (2026). **Semester/Exam Project—ETTML-01 Tiny Machine Learning** [projektbrief]. Brightspace, lokal PDF-kopi.
- [3] Analog Devices. (2021). **ADXL343: 3-axis, ± 2 g/ ± 4 g/ ± 8 g/ ± 16 g digital accelerometer** (Rev. A). [Datablad](#).
- [4] Particle Industries. (2026). **Photon 2 datasheet**. [Particle documentation](#).
- [5] Scikit-learn Developers. (2026). **StandardScaler**. [API documentation](#).
- [6] Scikit-learn Developers. (2026). **Neural network models (supervised): MLP classification**. [User guide](#).
- [7] Géron, A. (2022). **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow** (3. udg.). O'Reilly Media.
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [9] Raschka, S., Liu, Y., & Mirjalili, V. (2022). **Machine Learning with PyTorch and Scikit-Learn**. Packt.
- [10] Blackbox AI Technologies Inc. (2026). **BlackBox AI—Agents and developer tools**. [Officiel hjemmeside](#).
- [11] OpenAI. (2026). **Codex—AI coding agent**. [Officiel OpenAI-dokumentation](#).

14 Appendiks A: Reproduktion

Kommandoerne køres fra repository-rod:

```
source .venv/bin/activate
```

```
python Product/ml/randomized_capture_gui.py --port /dev/ttyACM1 --series 5
```

```
python Product/ml/train.py --config Product/ml/config.yaml
```

```
python Product/ml/export_model.py --config Product/ml/config.yaml
```

```
python Product/ml/generate_figures.py
```

```
particle compile photon2 Product/firmware --saveTo firmware.bin
```

```
particle flash TinyML_Node1 firmware.bin
```

LIVE aktiveres med `MODE LIVE`; `STATUS` kontrollerer mode, sensor, buffer, inference-count/timing og read errors.

15 Appendiks B: Kodearkitektur og sporbarhed

Fil/modul	Ansvar og vigtig invariant	Rapportafsnit
firmware/src/main.cpp	I2C, 400 Hz buffer, features, modes, decision-filter, LED. Sampling må ikke blokkes af feedback.	4, 6, 7
firmware/src/model_data.*	Genererede scaler-/MLP-konstanter og float32-forward-pass. Feature count skal være 28.	6
ml/randomized_capture_gui.py	Randomiseret GUI, serial capture, kvalitetstjek, retry og sidecars.	5
ml/train.py	Schema-normalisering, vinduer, features, split og metrikker.	6, 7
ml/export_model.py	Fail-closed atomisk eksport til C++.	6
ml/generate_figures.py	Figurer genberegnes fra aktuelle data og split.	5, 7
ml/config.yaml	Enkelt konfigurationspunkt for labels, sampling, model og decision logic.	5–7

Den centrale firmwaresekvens kan læses som følgende pseudokode:

loop:

```
parse serial command
if TRAINING: run capture state machine
advance non-blocking LED
if sample due: read synchronized XYZ into ringbuffer
if window ready:
    features = stat_v2(window)
    scores = scaler + MLP + softmax(features)
    candidate = confidence/smoothing(scores)
    candidate = impact guard(candidate, window)
    if stable LIVE event: emit serial + LED pattern
```

Kommentarer i source forklarer kontrakter og begrundelser—ikke blot hvad hver C++-linje gør. Genereret `model_data.cpp` kommenteres ved generatoren, så man undgår håndredigering af modelparametre.

16 Appendiks C: Datakontrakt

CSV-felterne er `time_us`, `x_g`, `y_g`, `z_g`, afledt tid/delta/magnitude, `label`, `requested pace/force`, `session_id` og `operator_accepted`. En accepteret fil har præcis 1.600 samples. JSON-sidecars er audit metadata; PNG-sidecars er visuel evidens. Rejected-filer må ikke indgå i træning. Configens `raw_dir` peger eksplicit på den accepterede deployment, så gamle eller diagnostiske data ikke blandes ind implicit.

17 Appendiks D: Værktøjer og AI-assistance

Python 3.12, NumPy, pandas, scikit-learn, PyYAML, pyserial og Matplotlib blev brugt til data og modeltræning. Particle CLI/Device OS blev brugt til at bygge og lægge firmwaren på enheden, og Typst blev brugt til PDF'en.

BlackBox AI og OpenAI Codex blev brugt som AI-assisterter til forslag til kode, fejlsøgning, oprydning, dokumentation og sproglig redigering [10], [11]. De fysiske målinger, valg af gestusser og den endelige vurdering er mine. Forslag fra AI blev kontrolleret ved at læse koden, se på datasættet, køre træning og build samt afprøve Photon 2. AI-værktøjerne er derfor hjælpemidler og ikke kilder til projektets måleresultater. `MLPClassifier` og `StandardScaler` er dele af programmet og ikke AI-assisterter.